



BUILD YOUR OWN PROJECT (BYOP) REPORT

Hangman Game in Java

Course Name: Programming in Java

Course Code: 2006

Institution: VIT Bhopal University

Submitted By:

Name: Jainam Jain

Registration No: 25BAI11474

Branch: B.Tech CSE (AIML)

Submitted To:

Dr. Muthusundari S

School of Computing Science and Engineering (SCSE)

VIT Bhopal University

1. Introduction

Hangman is a classic word-guessing game where the player tries to find a hidden word by guessing one letter at a time. The player has a limited number of attempts to guess the full word.

This project is a console-based Hangman game created using Java. Instead of putting all the code in one long file, the project is divided into simple, organized Java classes using Object-Oriented Programming (OOP) concepts.

2. Problem Statement

Many simple terminal games have the following issues:

- All code is written inside a single file, making it hard to read and manage.
- If a user enters a number or symbol by mistake, the program crashes or deducts an attempt unfairly.
- Game scores disappear as soon as the game window closes.

This project solves these issues by organizing the code into separate classes, adding input checks so invalid entries do not waste attempts, and saving player scores to a text file.

3. Functional Requirements

The project has three main functional modules:

Player & Score Module:

- Asks for the player's name at the start.
- Calculates points when the player wins (attempts left $\times$ 10).
- Saves the player's name and score to scores.txt.
- Displays past scores from the file.

Game Logic & Words Module:

- Stores words in 3 difficulty levels: Easy (7 turns), Medium (5 turns), and Hard (4 turns).
- Picks a random word for each game.
- Shows the word as underscores (e.g., _ _ _ _ _) and reveals correct letters.
- Checks whether the player has won or run out of attempts.

Input Validation Module:

- Checks that the player enters only one letter at a time.
- Blocks numbers and symbols.
- Alerts the player if a letter was already guessed without deducting a turn.

4. Non-Functional Requirements

- Usability: Simple menus and clear messages on the screen.

- Reliability: Does not crash on bad inputs (like blank spaces or numbers).
- Efficiency: Runs fast with minimal computer memory.
- Maintainability: Written in simple, separated files so adding new features is easy.

5. System Architecture

The application is divided into three simple layers:

- User Interface (Main.java): Shows the menu, prints messages, and takes user input.
- Game Engine (Game.java, WordList.java, Validator.java): Runs the rules, tracks turns, and validates inputs.
- Data Storage (Player.java, ScoreBoard.java): Holds player details and reads/writes to scores.txt.

6. Design Diagrams

6.1 Use Case Diagram

- Player → Enters Name
- Player → Selects Difficulty
- Player → Guesses Letters
- Player → Views Past Scores
- Player → Exits Game

6.2 Workflow Diagram

```

[Start Game]
  |
  v
[Enter Name]
  |
  v
[Choose Menu Option]
|-- 1. Play Game --> [Choose Difficulty]
|                   --> [Guess Letters Loop]
|                   --> [Win/Loss] --> [Save Score]
|-- 2. View Scores -> [Read & Print scores.txt]
+-- 3. Exit -----> [Quit Application]

```

6.3 Sequence Diagram (Guessing a Letter)

1. Player types a letter in Main.
2. Main calls Validator to verify the letter.
3. If valid, Main calls Game to check if the letter is in the word.
4. If correct, Game reveals the letter. If wrong, Game reduces turns by 1.
5. If the word is completed, Main calls ScoreBoard to save the score.

6. Main prints the updated word and turns on screen.

6.4 Class Diagram

Class Name	What It Does	Key Methods / Fields
Player	Stores player name and score	getName(), addScore(points)
WordList	Holds word arrays for Easy, Medium, Hard	getWord(choice)
Validator	Checks inputs and duplicate guesses	isValidGuess(), isAlreadyGuessed()
Game	Manages turns, hidden word, and guesses	makeGuess(), getDisplayWord(), isGameOver()
ScoreBoard	Saves and reads scores from file	saveScore(), showScores()
Main	Entry point with menu and game loop	main()
TestGame	Verifies game logic automatically	main()

7. Design Decisions & Rationale

- Console Interface: Kept in terminal to focus on Java basics and OOP without complex GUI setup.
- Separate Files: Creating separate classes makes the code clean, easy to read, and modular.
- File Storage with scores.txt: Used simple file writing so scores are permanently saved on the computer.
- Forgiving Input Checks: Entering duplicate letters does not waste a turn, making the game fair and user-friendly.

8. Implementation Details

The project is built using standard Java:

- Player.java: Uses private variables with getter methods for clean data encapsulation.
- WordList.java: Uses java.util.Random to select words from simple String arrays.
- Validator.java: Uses Character.isLetter() to ensure inputs are alphabetic.
- Game.java: Uses a simple loop to check matching characters and reveals them in getDisplayWord().
- ScoreBoard.java: Uses FileWriter to append lines to scores.txt and Scanner to read them back.
- Main.java: Uses a while loop to navigate through the menu until the user chooses to exit.

9. Screenshots / Sample Output

Main Menu & Difficulty:

```
What's your name? Alex
Hi, Alex Time to play Hangman!
1. Game Time
2. See Scores
3. Exit Game
```

9. Screenshots / Sample Output (continued)

Gameplay Screenshot:

The screenshot displays an IDE window titled "Hangman-Project" with several tabs open: Welcome, Player.java, WordList.java, Validator.java, Game.java, ScoreBoard.java, TestGame.java, and Main.java. The "Main.java" tab is active, showing the following code:

```

1 public class Main {
2     public static void main(String[] args) throws InterruptedException {
3         scoreBoard.showScores();
4         System.out.println("Game initiated: " + points);
5     }
6 }
7
8 } else {
9     System.out.println("The game is over");
10    System.out.println("The correct word was: " + game.getWord());
11 }
12
13 } else if (choice.equals(0)) {
14     scoreBoard.showScores();
15 } else if (choice.equals(1)) {
16     System.out.println("See you later! " + player.getName() + "!");
17     running = false;
18 } else {
19     System.out.println("Oops! Choose again.");
20 }
21
22 }
23
24 scanner.close();
25 }
26 }
27
28 }

```

The "Output" tab shows the following text:

```

PS C:\Users\jainee\Downloads\Hangman-Project> & "C:\Program Files\Java\jdk-21.0.12\bin\java.exe" "-Dlog.level=INFO" "-cp" "C:\Users\jainee\AppData\Local\Temp\Code\User\workspaceStorage\7756c08b-6a4b-4b87-82b2-2a6738\ed8at_java\src\main\Hangman-Project\src\main\bin" "Hangman.Main"
What's your name? Shiva
Hello Shiva! It's time to play Hangman!

1. Game Time
2. See Scores
3. Exit Game
Make a choice: 1

Set difficulty:
1. Easy Level - 7 Attempts
2. Medium Level - 5 Attempts
3. Hard Level - 4 Attempts
Choice: 3
Make your guess.

Word: _ _ _ _ _
Guesses:
Attempts left: 4
Guess a letter: e
Word: _ _ e _ _
Guesses: e
Attempts left: 4
Guess a letter: a
Word: _ a _ _ _
Guesses: ea
Attempts left: 4
Guess a letter: w
Word: w a _ _ _
Guesses: ewa
Attempts left: 4
Guess a letter: t
Word: w a t _ _
Guesses: ewat
Attempts left: 4
Guess a letter: r

```

Make a choice: 1
 Set difficulty:
 1. Easy Level - 7 Attempts
 2. Medium Level - 5 Attempts
 3. Hard Level - 4 Attempts
 Choice: 2
 Make your guess.

Gameplay & Winning:

Word: _ _ _ _ _
 Guesses:
 Attempts left: 5
 Guess a letter: a
 Word: _ a _ a _ a
 Guesses: a
 Attempts left: 5
 Guess a letter: b
 Word: b a _ a _ a
 Guesses: ab
 Attempts left: 5
 Guess a letter: n
 Word: b a n a n a
 You did it
 Score achieved: 50

High Scores (scores.txt):

--- High Scores ---
 Alex - 50 points
 Rahul - 30 points

10. Testing Approach

A dedicated testing class (TestGame.java) was written to verify all main functions:

Test	What was tested	Expected Result	Status
Test 1	Initial word masking	Hidden word shows as underscores (_ _ _ _)	PASS
Test 2	Guessing a correct letter	Correct letter is revealed in place of _	PASS
Test 3	Guessing a wrong letter	Remaining attempts decrease by 1	PASS
Test 4	Input checking	Valid letters pass; numbers and symbols are blocked	PASS

11. Challenges Faced

- Handling Wrong Inputs: Initially, typing numbers caused errors. This was resolved by creating `Validator.isValidGuess()` to check that input is a single letter before processing.
- Duplicate Guesses: At first, guessing the same letter twice wasted an attempt. Added `Validator.isAlreadyGuessed()` to warn the user without deducting a turn.
- Menu Navigation: Using `Integer.parseInt` sometimes crashed if the user pressed enter on an empty line. Changed menu inputs to simple string checks ("1", "2", "3") to prevent crashes.

12. Learnings & Key Takeaways

- Learned how to divide a program into separate classes rather than writing everything in one file.
- Understood how to read and write data to files using Java File I/O.
- Learned the importance of defensive programming and validating user input.
- Gained experience in writing simple unit tests to verify program correctness.

13. Future Enhancements

- Add a graphical interface (GUI) using Java Swing or JavaFX.
- Add sound effects for correct and incorrect guesses.
- Add an online multiplayer mode where two players can play together.
- Connect to an online dictionary API to get new words automatically.

14. References

- Oracle Java Documentation: <https://docs.oracle.com/en/java/>
- GeeksforGeeks Java Tutorials: <https://www.geeksforgeeks.org/java/>
- W3Schools Java File Handling: https://www.w3schools.com/java/java_files.asp